

Transform Your .NET Development Code Generation & Analyzers in Action!

The compiler as your fastest — and strictest — teammate in the AI era.

The new reality

The new reality

- AI agents write code **faster than ever**

The new reality

- AI agents write code **faster than ever**
- But *faster* is not the same as *better*

The new reality

- AI agents write code **faster than ever**
- But *faster* is not the same as *better*
- Two forces of the **compiler** make the difference:

The new reality

- AI agents write code **faster than ever**
- But *faster* is not the same as *better*
- Two forces of the **compiler** make the difference:
 - **Source Generators** — let the compiler write the repetition

The new reality

- AI agents write code **faster than ever**
- But *faster* is not the same as *better*
- Two forces of the **compiler** make the difference:
 - **Source Generators** — let the compiler write the repetition
 - **Analyzers** — let the compiler enforce your rules

The problem with "fast"

The problem with "fast"

- Agents duplicate boilerplate in **seconds**

The problem with "fast"

- Agents duplicate boilerplate in **seconds**
- Copy-paste at scale → **drift**, inconsistency, subtle bugs

The problem with "fast"

- Agents duplicate boilerplate in **seconds**
- Copy-paste at scale → **drift**, inconsistency, subtle bugs
- 10 variants of "almost the same" → maintenance hell

The problem with "fast"

- Agents duplicate boilerplate in **seconds**
- Copy-paste at scale → **drift**, inconsistency, subtle bugs
- 10 variants of "almost the same" → maintenance hell
- More code = more surface for mistakes

The problem with "fast"

- Agents duplicate boilerplate in **seconds**
- Copy-paste at scale → **drift**, inconsistency, subtle bugs
- 10 variants of "almost the same" → maintenance hell
- More code = more surface for mistakes

AI makes it cheap to do the wrong things — at scale.

Two allies of the compiler

	Source Generator	Analyzer
Role	Writes code	Guards code
When	At build time	At build time and in the IDE
Payoff	One source, zero duplication	Rules no one can ignore
Output	<code>.g.cs</code> files	Warnings / build errors

Part 1

Source Generators

Let the compiler write the repetition

The idea: one attribute

```
[GeneratedEnum(EnumType.Int)]
public sealed partial class Priority
{
    public required string Name { get; init; }

    public static Priority Low { get; } = new() { Value = 1, Name = "Low" };
    public static Priority High { get; } = new() { Value = 2, Name = "High" };
}
```

The idea: one attribute

```
[GeneratedEnum(EnumType.Int)]
public sealed partial class Priority
{
    public required string Name { get; init; }

    public static Priority Low { get; } = new() { Value = 1, Name = "Low" };
    public static Priority High { get; } = new() { Value = 2, Name = "High" };
}
```

You write the **intent**. The compiler writes the rest.

What the compiler produces

```
// Priority.g.cs - auto-generated, never touched by hand
[JsonConverter(typeof(PriorityJsonConverter))]
public sealed partial class Priority : IntEnum<Priority>;

public class PriorityJsonConverter : JsonConverter<Priority>
{
    public override Priority Read(ref Utf8JsonReader r, Type t, JsonSerializerOptions o)
        => Priority.All.Single(x => x.Value == r.GetInt32());
    // + Write(...)
}

public class PrioritySqlConverter() : ValueConverter<Priority, int>(
    convertToProviderExpression: x => x.Value,
    convertFromProviderExpression: x => Priority.All.Single(a => a.Value == x)
);
```

What the compiler produces

```
// Priority.g.cs - auto-generated, never touched by hand
[JsonConverter(typeof(PriorityJsonConverter))]
public sealed partial class Priority : IntEnum<Priority>;

public class PriorityJsonConverter : JsonConverter<Priority>
{
    public override Priority Read(ref Utf8JsonReader r, Type t, JsonSerializerOptions o)
        => Priority.All.Single(x => x.Value == r.GetInt32());
    // + Write(...)
}

public class PrioritySqlConverter() : ValueConverter<Priority, int>(
    convertToProviderExpression: x => x.Value,
    convertFromProviderExpression: x => Priority.All.Single(a => a.Value == x)
);
```

System.Text.Json + Newtonsoft + EF converter — for *every* enum, for free.

Generator vs. AI duplication

	Source Generator	Agent that copies
Source of truth	One (the attribute)	Spread across N files
Sync	Always, automatic	Manual, drifts apart
Determinism	Identical on every build	Varies per prompt
Cost per new enum	0 tokens, 0 seconds	Tokens + review, every time
Add a new rule	Change 1 place	Chase down N places

Generator vs. AI duplication

	Source Generator	Agent that copies
Source of truth	One (the attribute)	Spread across N files
Sync	Always, automatic	Manual, drifts apart
Determinism	Identical on every build	Varies per prompt
Cost per new enum	0 tokens, 0 seconds	Tokens + review, every time
Add a new rule	Change 1 place	Chase down N places

The generator scales with one attribute. Duplication scales with your maintenance burden.

The generator is this small

```
[Generator]
internal sealed class EnumGenerator : IIncrementalGenerator
{
    public void Initialize(IncrementalGeneratorInitializationContext context)
    {
        var enums = context.SyntaxProvider.ForAttributeWithMetadataName(
            GeneratedEnumAttribute.Name,
            predicate: static (node, _) => node is ClassDeclarationSyntax,
            transform: static (ctx, _) => GetEnumToGenerate(ctx));

        context.RegisterSourceOutput(enums,
            static (ctx, e) => ctx.AddSource($"{e.ClassName}.g.cs", e.BuildCode()));
    }
}
```

The generator is this small

```
[Generator]
internal sealed class EnumGenerator : IIncrementalGenerator
{
    public void Initialize(IncrementalGeneratorInitializationContext context)
    {
        var enums = context.SyntaxProvider.ForAttributeWithMetadataName(
            GeneratedEnumAttribute.Name,
            predicate: static (node, _) => node is ClassDeclarationSyntax,
            transform: static (ctx, _) => GetEnumToGenerate(ctx));

        context.RegisterSourceOutput(enums,
            static (ctx, e) => ctx.AddSource($"{e.ClassName}.g.cs", e.BuildCode()));
    }
}
```

This kind of code is **fine to let an agent write** — it's small, standardized, and the compiler tells you immediately whether it's right.

Part 2

Analyzers

Let the compiler enforce your rules

Instructions vs. enforcement

Instructions vs. enforcement

- Conventions in a README, CLAUDE.md or copilot-instructions

Instructions vs. enforcement

- Conventions in a README, CLAUDE.md or copilot-instructions
- ... get read **when it's convenient**

Instructions vs. enforcement

- Conventions in a README, CLAUDE.md or copilot-instructions
- ... get read **when it's convenient**
- An agent can **ignore, forget or "creatively interpret"** an instruction

Instructions vs. enforcement

- Conventions in a README, CLAUDE.md or copilot-instructions
- ... get read **when it's convenient**
- An agent can **ignore, forget or "creatively interpret"** an instruction

***AI agents can ignore instructions.
Build errors, they can't.***

Analyzer 1 — a naming convention

```
[DiagnosticAnalyzer(LanguageNames.CSharp)]
public class TypoAnalyzer : DiagnosticAnalyzer // APP1001
{
    // "Adress" -> did you mean "Address"?
    public override void Initialize(AnalysisContext context)
    {
        context.EnableConcurrentExecution();
        context.RegisterSymbolAction(AnalyzeSymbol,
            SymbolKind.Property, SymbolKind.Field, SymbolKind.NamedType);
    }

    static void AnalyzeSymbol(SymbolAnalysisContext ctx)
    {
        if (ctx.Symbol.Name.Contains("Adress", StringComparison.OrdinalIgnoreCase))
            ctx.ReportDiagnostic(Diagnostic.Create(Rule, ctx.Symbol.Locations[0], ctx.Symbol.Name));
    }
}
```

Analyzer 1 — a naming convention

```
[DiagnosticAnalyzer(LanguageNames.CSharp)]
public class TypoAnalyzer : DiagnosticAnalyzer // APP1001
{
    // "Adress" -> did you mean "Address"?
    public override void Initialize(AnalysisContext context)
    {
        context.EnableConcurrentExecution();
        context.RegisterSymbolAction(AnalyzeSymbol,
            SymbolKind.Property, SymbolKind.Field, SymbolKind.NamedType);
    }

    static void AnalyzeSymbol(SymbolAnalysisContext ctx)
    {
        if (ctx.Symbol.Name.Contains("Adress", StringComparison.OrdinalIgnoreCase))
            ctx.ReportDiagnostic(Diagnostic.Create(Rule, ctx.Symbol.Locations[0], ctx.Symbol.Name));
    }
}
```

From error to fix

From error to fix

- An analyzer **points at the problem** — a **code fix** solves it

From error to fix

- An analyzer **points at the problem** — a **code fix** solves it
- One click via the light bulb  , or automatically via **dotnet format**

From error to fix

- An analyzer **points at the problem** — a **code fix** solves it
- One click via the light bulb  , or automatically via **dotnet format**
- The fix **is** the correct form — perfect for an agent, no guesswork

From error to fix

- An analyzer **points at the problem** — a **code fix** solves it
- One click via the light bulb  , or automatically via **dotnet format**
- The fix **is** the correct form — perfect for an agent, no guesswork

```
public string Address { get; set; }  
~~~~~ APP1001: did you mean 'Address'?  
 Rename to 'Address' ← one click
```

The code fix (APP1001)

```
[ExportCodeFixProvider(LanguageNames.CSharp)]
public sealed class TypoCodeFixProvider : CodeFixProvider
{
    public override ImmutableArray<string> FixableDiagnosticIds => ["APP1001"];

    public override async Task RegisterCodeFixesAsync(CodeFixContext context)
    {
        var root = await context.Document.GetSyntaxRootAsync(context.CancellationToken);
        var node = root!.FindToken(context.Span.Start).Parent!;

        context.RegisterCodeFix(
            CodeAction.Create(
                title: "Rename to 'Address'",
                createChangedSolution: c => RenameAsync(context.Document, node, c),
                equivalenceKey: "RenameToAddress"),
            context.Diagnostics[0]);
    }
}
```

The code fix (APP1001)

```
[ExportCodeFixProvider(LanguageNames.CSharp)]
public sealed class TypoCodeFixProvider : CodeFixProvider
{
    public override ImmutableArray<string> FixableDiagnosticIds => ["APP1001"];

    public override async Task RegisterCodeFixesAsync(CodeFixContext context)
    {
        var root = await context.Document.GetSyntaxRootAsync(context.CancellationToken);
        var node = root!.FindToken(context.Span.Start).Parent!;

        context.RegisterCodeFix(
            CodeAction.Create(
                title: "Rename to 'Address'",
                createChangedSolution: c => RenameAsync(context.Document, node, c),
                equivalenceKey: "RenameToAddress"),
            context.Diagnostics[0]);
    }
}
```

Code fixes belong in a **separate project** — they reference **Microsoft.CodeAnalysis.Workspaces** (which the analyzer itself must not: **RS1038**).

 **Live demo**

Live demo

- A **Customer** with an **Adress** typo → **the build fails** (APP1001 as an error)

Live demo

- A **Customer** with an **Adress** typo → **the build fails** (APP1001 as an error)
- I fix it live: 💡 **Rename to 'Address'** → build goes green

Live demo

- A `Customer` with an `Adress` typo → **the build fails** (APP1001 as an error)
- I fix it live: 💡 **Rename to 'Address'** → build goes green
- And the generated `Priority.g.cs` — the compiler wrote the `Value`

Live demo

- A **Customer** with an **Adress** typo → **the build fails** (APP1001 as an error)
- I fix it live: 💡 **Rename to 'Address'** → build goes green
- And the generated **Priority.g.cs** — the compiler wrote the **Value**

```
examples/ -> dotnet build ❌ error APP1001  
fix + rebuild ✅ High (value 2)
```

Analyzer 2 — an architecture rule

```
[DiagnosticAnalyzer(LanguageNames.CSharp)]
public class DbContextAnalyzer : DiagnosticAnalyzer    // APP2001
{
    // "Constructor takes a DbContext -> use IDbContextFactory"
    public override void Initialize(AnalysisContext context)
    {
        context.RegisterSymbolAction(AnalyzeSymbol, SymbolKind.Method);
    }

    static void AnalyzeSymbol(SymbolAnalysisContext ctx)
    {
        if (ctx.Symbol is IMethodSymbol { MethodKind: MethodKind.Constructor } c &&
            c.Parameters.Any(p => p.Type.Is("Microsoft.EntityFrameworkCore.DbContext")))
            ctx.ReportDiagnostic(Diagnostic.Create(Rule, c.Locations[0], c.ContainingType.Name));
    }
}
```

Analyzer 2 — an architecture rule

```
[DiagnosticAnalyzer(LanguageNames.CSharp)]
public class DbContextAnalyzer : DiagnosticAnalyzer    // APP2001
{
    // "Constructor takes a DbContext -> use IDbContextFactory"
    public override void Initialize(AnalysisContext context)
    {
        context.RegisterSymbolAction(AnalyzeSymbol, SymbolKind.Method);
    }

    static void AnalyzeSymbol(SymbolAnalysisContext ctx)
    {
        if (ctx.Symbol is IMethodSymbol { MethodKind: MethodKind.Constructor } c &&
            c.Parameters.Any(p => p.Type.Is("Microsoft.EntityFrameworkCore.DbContext")))
            ctx.ReportDiagnostic(Diagnostic.Create(Rule, c.Locations[0], c.ContainingType.Name));
    }
}
```

From suggestion to guarantee

Layered defence — each layer stricter than the last:

From suggestion to guarantee

Layered defence — each layer stricter than the last:

1. **copilot-instructions.md** — broad context (*can be ignored*)

From suggestion to guarantee

Layered defence — each layer stricter than the last:

1. **copilot-instructions.md** — broad context (*can be ignored*)
2. **.claude/rules/** — path-scoped MUST/NEVER (*always loaded on edit*)

From suggestion to guarantee

Layered defence — each layer stricter than the last:

1. **copilot-instructions.md** — broad context (*can be ignored*)
2. **.claude/rules/** — path-scoped MUST/NEVER (*always loaded on edit*)
3. **Analyzer** — warning in the IDE (*visible, live*)

From suggestion to guarantee

Layered defence — each layer stricter than the last:

1. **copilot-instructions.md** — broad context (*can be ignored*)
2. **.claude/rules/** — path-scoped MUST/NEVER (*always loaded on edit*)
3. **Analyzer** — warning in the IDE (*visible, live*)
4. **TreatWarningsAsErrors** in CI — **build fails** (*cannot be bypassed*)

From suggestion to guarantee

Layered defence — each layer stricter than the last:

1. **copilot-instructions.md** — broad context (*can be ignored*)
2. **.claude/rules/** — path-scoped MUST/NEVER (*always loaded on edit*)
3. **Analyzer** — warning in the IDE (*visible, live*)
4. **TreatWarningsAsErrors** in CI — **build fails** (*cannot be bypassed*)

*Instructions persuade. The analyzer + CI **guarantee**.*

Part 3

Performance

The warning your agent doesn't feel

Why performance is critical

Why performance is critical

- Analyzers are injected into **every** project

Why performance is critical

- Analyzers are injected into **every** project
- They run on **every build** — and **continuously in the IDE**

Why performance is critical

- Analyzers are injected into **every** project
- They run on **every build** — and **continuously in the IDE**
- A slow analyzer runs on **every keystroke**, across the whole solution

Why performance is critical

- Analyzers are injected into **every** project
- They run on **every build** — and **continuously in the IDE**
- A slow analyzer runs on **every keystroke**, across the whole solution

*One sloppy analyzer taxes the IDE responsiveness and the build time of the **entire team**.*

Slow vs. fast

```
1 // ❌ SLOW: runs for every node in the whole solution, allocates per node
2 context.RegisterSymbolAction(ctx =>
3 {
4     if (ctx.Symbol.ToDisplayString() == "MyApp.Marker")    // string per node!
5         ...
6 }, SymbolKind.NamedType);
7
8 // ✅ FAST: scope once per compilation, compare by symbol
9 context.RegisterCompilationStartAction(start =>
10 {
11     var marker = start.Compilation.GetTypeByMetadataName("MyApp.Marker");
12     if (marker is null) return;
13     // irrelevant projects: 0 work
14
15     start.RegisterSymbolAction(ctx => {
16         if (SymbolEqualityComparer.Default.Equals(ctx.Symbol, marker)) ...
17     }, SymbolKind.NamedType);
18 });
```

Slow vs. fast

```
1 // ❌ SLOW: runs for every node in the whole solution, allocates per node
2 context.RegisterSymbolAction(ctx =>
3 {
4     if (ctx.Symbol.ToDisplayString() == "MyApp.Marker")    // string per node!
5         ...
6 }, SymbolKind.NamedType);
7
8 // ✅ FAST: scope once per compilation, compare by symbol
9 context.RegisterCompilationStartAction(start =>
10 {
11     var marker = start.Compilation.GetTypeByMetadataName("MyApp.Marker");
12     if (marker is null) return;
13     // irrelevant projects: 0 work
14
15     start.RegisterSymbolAction(ctx => {
16         if (SymbolEqualityComparer.Default.Equals(ctx.Symbol, marker)) ...
17     }, SymbolKind.NamedType);
18 });
```

The checklist for fast analyzers

The checklist for fast analyzers

- **Scope once** in `RegisterCompilationStartAction` — irrelevant projects do 0 work

The checklist for fast analyzers

- **Scope once** in `RegisterCompilationStartAction` — irrelevant projects do 0 work
- **Resolve marker types once** via `GetTypeByMetadataName`, compare with `SymbolEqualityComparer`

The checklist for fast analyzers

- **Scope once** in `RegisterCompilationStartAction` — irrelevant projects do 0 work
- **Resolve marker types once** via `GetTypeByMetadataName`, compare with `SymbolEqualityComparer`
- **Never** `ISymbol.ToString()` per node — it allocates every time

The checklist for fast analyzers

- **Scope once** in `RegisterCompilationStartAction` — irrelevant projects do 0 work
- **Resolve marker types once** via `GetTypeByMetadataName`, compare with `SymbolEqualityComparer`
- **Never** `ISymbol.ToString()` per node — it allocates every time
- `EnableConcurrentExecution()` + `ConfigureGeneratedCodeAnalysis(None)`

The checklist for fast analyzers

- **Scope once** in `RegisterCompilationStartAction` — irrelevant projects do 0 work
- **Resolve marker types once** via `GetTypeByMetadataName`, compare with `SymbolEqualityComparer`
- **Never** `ISymbol.ToString()` per node — it allocates every time
- `EnableConcurrentExecution()` + `ConfigureGeneratedCodeAnalysis(None)`
- **No** per-compilation state in analyzer fields (RS1008)

Generators must be fast too

Generators must be fast too

- Use **IIncrementalGenerator** — not the old **ISourceGenerator**

Generators must be fast too

- Use **IIncrementalGenerator** — not the old **ISourceGenerator**
- **ForAttributeWithMetadataName** → the pipeline only runs on relevant nodes

Generators must be fast too

- Use **IIncrementalGenerator** — not the old **ISourceGenerator**
- **ForAttributeWithMetadataName** → the pipeline only runs on relevant nodes
- Model as a **value type / equatable record** → the compiler caches and doesn't regenerate needlessly

Generators must be fast too

- Use **IIncrementalGenerator** — not the old **ISourceGenerator**
- **ForAttributeWithMetadataName** → the pipeline only runs on relevant nodes
- Model as a **value type / equatable record** → the compiler caches and doesn't regenerate needlessly
- Keep the **transform** step **pure and small**

Generators must be fast too

- Use **IIncrementalGenerator** — not the old **ISourceGenerator**
- **ForAttributeWithMetadataName** → the pipeline only runs on relevant nodes
- Model as a **value type / equatable record** → the compiler caches and doesn't regenerate needlessly
- Keep the **transform** step **pure and small**

*A non-incremental generator re-runs on every keystroke
→ IDE lag.*

Make your agent aware of this

Make your agent aware of this

- An agent **doesn't feel IDE lag** — it only sees "compiles / doesn't compile"

Make your agent aware of this

- An agent **doesn't feel IDE lag** — it only sees "compiles / doesn't compile"
- Without direction it writes a correct, but **slow** analyzer

Make your agent aware of this

- An agent **doesn't feel IDE lag** — it only sees "compiles / doesn't compile"
- Without direction it writes a correct, but **slow** analyzer
- Performance is an **explicit review item**, not an afterthought

Make your agent aware of this

- An agent **doesn't feel IDE lag** — it only sees "compiles / doesn't compile"
- Without direction it writes a correct, but **slow** analyzer
- Performance is an **explicit review item**, not an afterthought

*The agent optimizes for what you hold it accountable to.
Hold it accountable for performance.*

Part 4

The AI Skill

Teach the agent to do it right from the start

What is a skill?

What is a skill?

- A reusable **instruction package** the agent loads for a matching task

What is a skill?

- A reusable **instruction package** the agent loads for a matching task
- Lives **in the repo**, in git → shared with the whole team and every agent

What is a skill?

- A reusable **instruction package** the agent loads for a matching task
- Lives **in the repo**, in git → shared with the whole team and every agent
- Captures the **how** and the **pitfalls** — once, for good

What is a skill?

- A reusable **instruction package** the agent loads for a matching task
- Lives **in the repo**, in git → shared with the whole team and every agent
- Captures the **how** and the **pitfalls** — once, for good

Don't re-explain it in every prompt. The knowledge lives next to the code.

The skill in practice

name: authoring-roslyn-tools description: Use when creating or changing a Roslyn analyzer, code fix, or source generator. Covers the required perf checklist, because analyzers run solution-wide on every build and keystroke.

Analyzers & generators — do it right

**Analyzer performance (always
review)**

- Gate scope once in RegisterCompilationStartAction
- Resolve marker types once (GetTypeByMetadataName), compare by symbol
- NEVER call ISymbol.ToDisplayString() per node
- EnableConcurrentExecution +
ConfigureGeneratedCodeAnalysis(None)

Generators

- IncrementalGenerator + ForAttributeWithMetadataName
- Model = equatable readonly record struct (cacheable)

```
<p class="fragment fade-in">The full SKILL.md ships alongside these slides.</p>
```

```
<aside class="notes"><p>This is the bridge: everything from part 3 gets locked in here structurally. The agent reads it automatically the moment it touches an analyzer or generator.</p></aside>
```

In summary

In summary

- **Generate > duplicate** — one source, zero drift, scales for free

In summary

- **Generate > duplicate** — one source, zero drift, scales for free
- **Analyzers > instructions** — a build error can't be ignored by anyone

In summary

- **Generate > duplicate** — one source, zero drift, scales for free
- **Analyzers > instructions** — a build error can't be ignored by anyone
- **Let the agent write the code** — but guard the **choices** and the **performance**

In summary

- **Generate > duplicate** — one source, zero drift, scales for free
- **Analyzers > instructions** — a build error can't be ignored by anyone
- **Let the agent write the code** — but guard the **choices** and the **performance**
- **Bake your rules into a skill** — knowledge that travels with the code

In summary

- **Generate > duplicate** — one source, zero drift, scales for free
- **Analyzers > instructions** — a build error can't be ignored by anyone
- **Let the agent write the code** — but guard the **choices** and the **performance**
- **Bake your rules into a skill** — knowledge that travels with the code

In the AI era, the compiler is your most reliable teammate.

Thank you!

Questions?

*Transform Your .NET Development: Code Generation & Analyzers
in Action*